

Quiz — JavaScript Fundamentos (Parts 01–04)

P1. Qual foi a principal motivação para a criação do standard ECMAScript?

- A) Substituir o Java nos navegadores web
- B) Garantir uniformidade do comportamento do JavaScript entre diferentes navegadores
- C) Introduzir tipagem estática na linguagem
- D) Separar o JavaScript do ambiente Node.js

✅ **Resposta correta:** B — O ECMAScript foi estabelecido precisamente para assegurar que o JavaScript se comportasse de forma consistente nos vários browsers, eliminando divergências de implementação entre fabricantes.

P2. O que acontece quando se utiliza o operador `+` entre uma string e um número em JavaScript?

- A) Ocorre um erro de tipo em tempo de execução
- B) O número é convertido para string e são concatenados
- C) A string é convertida para número e são somados
- D) O resultado é sempre `NaN`

✅ **Resposta correta:** B — JavaScript aplica coerção de tipos implícita: quando um dos operandos do `+` é string, o outro é convertido para string e o resultado é uma concatenação. Por exemplo, `"5" + 1` produz `"51"`.

P3. Qual é a diferença entre `==` e `===` em JavaScript?

- A) `===` compara apenas o tipo, enquanto `==` compara valor e tipo
- B) `==` permite coerção de tipos antes de comparar; `===` exige que valor e tipo sejam idênticos
- C) Não existe diferença prática entre os dois operadores
- D) `===` só funciona com valores primitivos, enquanto `==` funciona com objetos

✅ **Resposta correta:** B — O operador `==` é de igualdade "solta" e converte automaticamente os operandos para um tipo compatível antes de comparar (e.g., `false == 0` é `true`). O operador `===` é de igualdade estrita e não permite qualquer coerção, pelo que `false === 0` é `false`.

P4. Qual das seguintes afirmações sobre `null` e `undefined` em JavaScript é correta?

- A) `typeof null` retorna `"null"`
- B) `typeof undefined` retorna `"object"`
- C) `typeof null` retorna `"object"` — um comportamento quirky reconhecido da linguagem
- D) `null` e `undefined` são o mesmo valor com nomes diferentes

✅ **Resposta correta:** C — `typeof null` retorna `"object"`, o que é considerado uma incorreção histórica conhecida da linguagem. `typeof undefined` retorna corretamente `"undefined"`. Os dois valores têm

semânticas distintas: `undefined` indica uma variável declarada sem valor atribuído; `null` representa explicitamente "sem valor".

P5. Qual é o comportamento do `var` quando declarado dentro de um bloco `if`?

- A) Fica com escopo local ao bloco `if`, tal como `let`
- B) Fica com escopo global ou de função, ignorando o bloco
- C) Lança um erro por não ser permitido dentro de blocos
- D) Fica acessível apenas dentro do bloco onde foi declarado

✅ **Resposta correta:** B — O `var` tem escopo de função ou global, nunca de bloco. Uma variável declarada com `var` dentro de um `if` é acessível fora desse bloco, ao contrário de `let`, que respeita o escopo de bloco.

P6. Num ciclo `do-while`, em que momento é avaliada a condição de continuação?

- A) Antes da primeira execução do bloco
- B) No início de cada iteração, incluindo a primeira
- C) Apenas depois da primeira execução do bloco, e depois no final de cada iteração seguinte
- D) A condição é avaliada uma única vez, no final

✅ **Resposta correta:** C — No `do-while`, o corpo do ciclo é sempre executado pelo menos uma vez antes de qualquer avaliação da condição. A condição é verificada no final de cada iteração, determinando se o ciclo continua.

P7. Num `switch`, o que acontece se não se colocar `break` no final de um `case`?

- A) O programa termina com erro
- B) Apenas o `case` correspondente é executado e o switch termina
- C) A execução "cai" para o(s) case(s) seguinte(s) até encontrar um `break` ou o fim do switch
- D) O `default` é executado imediatamente

✅ **Resposta correta:** C — Sem `break`, o JavaScript continua a executar o código dos `case` seguintes em sequência, independentemente de o valor corresponder ou não. Este comportamento chama-se *fall-through*.

P8. Qual das seguintes afirmações descreve corretamente a "Declaration Notation" de funções em JavaScript?

- A) A função só pode ser chamada depois da linha onde foi declarada
- B) A função pode ser chamada antes da sua declaração no código, graças ao *hoisting*
- C) A função declarada desta forma não pode ter parâmetros
- D) A função retorna sempre `null` se não tiver `return`

✅ **Resposta correta:** B — Funções definidas com Declaration Notation são alvo de *hoisting*: o motor JavaScript move a sua declaração para o topo do escopo antes da execução, permitindo chamá-las antes de aparecerem no código fonte. Sem `return`, a função devolve `undefined`, não `null`.

P9. Qual é a forma mais compacta de escrever uma arrow function que recebe um argumento e retorna o seu quadrado?

- A) `const square = function(x) { return x * x; }`
- B) `const square = (x) => { return x * x; }`
- C) `const square = x => x * x;`
- D) `const square = x { return x * x; }`

✅ **Resposta correta:** C — Numa arrow function com um único parâmetro, os parênteses podem ser omitidos. Se o corpo for uma única expressão, as chavetas e o `return` também podem ser omitidos, tornando a sintaxe mais compacta.

P10. O que é um *closure* em JavaScript?

- A) Uma função que não aceita argumentos externos
- B) Uma função que referencia variáveis do escopo local que a envolve, mesmo após esse escopo ter terminado
- C) Uma função declarada dentro de outra função que não tem acesso a variáveis externas
- D) Um mecanismo que impede a reutilização de variáveis entre funções

✅ **Resposta correta:** B — Um *closure* ocorre quando uma função interna mantém referência a variáveis do escopo em que foi criada, mesmo depois de esse escopo ter terminado a sua execução. A função continua a "lembrar" e a poder manipular essas variáveis.

P11. O que acontece ao *call stack* quando uma função recursiva não tem condição de paragem?

- A) O programa entra em pausa até o utilizador intervir
- B) O garbage collector liberta a memória automaticamente
- C) A pilha de chamadas enche até esgotar a memória, resultando num erro de *stack overflow*
- D) A função é automaticamente convertida numa iteração pelo motor JavaScript

✅ **Resposta correta:** C — Cada chamada a uma função armazena o contexto de execução na *call stack*. Sem condição de paragem, as chamadas acumulam-se indefinidamente até a memória se esgotar, lançando um erro do tipo `RangeError: Maximum callstack size exceeded`.

P12. Qual é a diferença entre os valores de tipo primitivo e os de tipo objeto relativamente à mutabilidade?

- A) Tanto primitivos como objetos são imutáveis em JavaScript
- B) Primitivos são imutáveis; objetos são mutáveis (as suas propriedades podem ser alteradas)
- C) Objetos são imutáveis; primitivos podem ser alterados diretamente
- D) A mutabilidade depende de como a variável foi declarada (`let` vs `const`)

✅ **Resposta correta:** B — Os valores primitivos (número, string, boolean, etc.) não podem ser alterados na memória — apenas se pode atribuir um novo valor à variável. Os objetos, pelo contrário, são mutáveis: as suas propriedades podem ser adicionadas, modificadas ou removidas mesmo que a variável seja `const`.

P13. O que faz o operador `in` quando aplicado a um objeto?

- A) Itera sobre todos os valores do objeto
- B) Verifica se um determinado nome de propriedade existe no objeto, retornando um booleano
- C) Remove uma propriedade do objeto
- D) Copia as propriedades de um objeto para outro

✓ **Resposta correta:** B — O operador `in` recebe uma string com o nome de uma propriedade e um objeto, e retorna `true` se essa propriedade existir no objeto, `false` caso contrário. Ex.: `'color' in point1`.

P14. Numa *constructor function*, o que representa `this`?

- A) O objeto global `window` ou `globalThis`
- B) A própria função construtora
- C) O novo objeto que está a ser criado pela invocação com `new`
- D) O objeto protótipo associado à função

✓ **Resposta correta:** C — Quando uma função é invocada com o operador `new`, JavaScript cria automaticamente um novo objeto e faz com que `this` dentro da função aponte para esse novo objeto. As propriedades atribuídas a `this` passam a pertencer ao objeto instanciado.

P15. Qual é a utilidade do `prototype` numa *constructor function*?

- A) Definir propriedades privadas que não podem ser acedidas por instâncias
- B) Adicionar propriedades e métodos que ficam disponíveis em todas as instâncias criadas pela função
- C) Impedir que novas instâncias sejam criadas após a primeira
- D) Guardar o histórico de todas as instâncias criadas

✓ **Resposta correta:** B — O `prototype` é um objeto partilhado por todas as instâncias criadas por uma *constructor function*. Ao adicionar propriedades ou métodos ao `prototype`, esses ficam automaticamente acessíveis em todas as instâncias existentes e futuras.

P16. Qual é o resultado de `[0, 1, 2, 3, 4].slice(2, 4)` em JavaScript?

- A) `[0, 1]`
- B) `[2, 3, 4]`
- C) `[2, 3]`
- D) `[3, 4]`

✓ **Resposta correta:** C — O método `slice(start, end)` retorna um novo array com os elementos desde o índice `start` (inclusive) até `end` (exclusive). `slice(2, 4)` devolve os elementos nos índices 2 e 3, ou seja, `[2, 3]`.

P17. Qual a diferença entre os métodos `call` e `apply` quando se invoca uma função com um objeto explícito para `this`?

- A) `call` passa os argumentos como array; `apply` passa-os individualmente
- B) `apply` passa os argumentos como array; `call` passa-os individualmente

- C) `call` só funciona com funções sem argumentos; `apply` funciona com qualquer função
- D) Não existe diferença funcional — são aliases um do outro

✅ **Resposta correta:** B — Ambos permitem definir explicitamente o valor de `this` e invocar a função. A distinção está na forma como os argumentos adicionais são passados: `call` recebe-os separados por vírgula, enquanto `apply` exige que sejam passados num array.

P18. O que faz o operador de optional chaining (`?.`) em JavaScript?

- A) Lança uma exceção controlada se a propriedade não existir
- B) Acede a uma propriedade e retorna `undefined` em vez de erro se a propriedade ou o objeto não existir
- C) Verifica se um objeto tem uma propriedade antes de a criar
- D) Converte automaticamente `null` para um objeto vazio `{}`

✅ **Resposta correta:** B — O `?.` é uma variante do acesso por ponto que interrompe a cadeia de acesso e retorna `undefined` silenciosamente quando o valor à esquerda é `null` ou `undefined`, evitando erros em tempo de execução.

P19. O que é a *destructuring* de objetos e qual o seu efeito prático?

- A) Elimina todas as propriedades de um objeto, deixando-o vazio
- B) Cria uma cópia profunda (*deep copy*) do objeto
- C) Extrai valores de um objeto para variáveis individuais com o mesmo nome das propriedades
- D) Converte um objeto em array para facilitar a iteração

✅ **Resposta correta:** C — A *destructuring* de objetos permite declarar variáveis cujos nomes correspondem a propriedades do objeto, atribuindo-lhes automaticamente os respetivos valores. Por exemplo, `const { x, y } = obj` cria as variáveis `x` e `y` com os valores de `obj.x` e `obj.y`.

P20. Qual o propósito de `JSON.stringify` e `JSON.parse` e em que situações são especialmente úteis?

- A) `stringify` valida a estrutura de um objeto; `parse` converte-o para XML
- B) `stringify` serializa um objeto JavaScript para uma string de texto; `parse` faz o processo inverso — ambos são úteis para transmitir ou armazenar dados
- C) `stringify` comprime um objeto para ocupar menos memória; `parse` descomprime-o
- D) `stringify` converte um objeto em array; `parse` converte um array em objeto

✅ **Resposta correta:** B — `JSON.stringify` transforma um objeto JavaScript numa representação textual em formato JSON (serialização), útil para enviar dados em rede ou guardar em ficheiro. `JSON.parse` faz o inverso, reconstruindo o objeto a partir da string. Isto é especialmente necessário porque objetos com referências internas podem não estar em espaço de memória contíguo.
